Resumo – Arquivos em Python

1. A ideia central

Todo programa tem **entrada**, **processamento** e **saída**. Até agora, a entrada vinha do teclado (`input`) e a saída ia para a tela (`print`). Entre os dois, tudo ficava na **memória**: variáveis, listas e dicionários.

O problema: a memória é **volátil**. Quando o programa termina, tudo o que estava nela desaparece. Para continuar amanhã de onde paramos, os dados precisam ir para o **disco**.

	Memória (RAM)	**Armazenamento (disco)**
guarda	variáveis, listas, dicionários	arquivos
dura até	o programa terminar	alguém apagar
velocidade	muito rápida	mais lenta
capacidade	menor	muito maior

Um **arquivo** é uma coleção de dados **persistentes**, guardada em um dispositivo de armazenamento secundário. Ele sobrevive ao fim do programa, ao desligamento do computador, e pode ser lido por outro programa, em outro dia.

Macete

O arquivo é só um novo canal de entrada e saída: **ler de um arquivo** faz o papel do `input()` e **escrever em um arquivo** faz o papel do `print()`. A diferença é que o resultado fica guardado.

2. O ciclo de vida: abrir, processar, fechar

Todo código que mexe com arquivos segue três passos, sempre nesta ordem:

1. **Abrir** (`open`): cria uma ponte entre o programa e o arquivo;
2. **Processar** (ler ou escrever): transfere os dados pela ponte;
3. **Fechar** (`close`): corta a ponte, salvando as alterações e liberando os recursos.

```
# Abrindo no modo de escrita
arquivo = open("exemplo.txt", "w", encoding="utf-8")

# Escrevendo algo
arquivo.write("Ola, Mundo!")

# Fechando o arquivo
arquivo.close()
```

A função open, parâmetro por parâmetro

`"exemplo.txt"`	**Nome do arquivo.** Pode incluir o caminho, como `"dados/exemplo.txt"`.
`"w"`	**Modo de abertura.** Indica se o arquivo será aberto para leitura ou para escrita.
`encoding="utf-8"`	**Codificação.** Parâmetro opcional, mas de uso altamente recomendado.

Os modos de abertura

Modo	Significado	Comportamento
`"r"`	leitura (*read*)	o arquivo **precisa existir**; caso contrário, erro
`"w"`	escrita (*write*)	**apaga todo o conteúdo** existente, ou cria o arquivo
`"a"`	adição (*append*)	escreve **a partir do final**, sem apagar nada

Macete

Para decorar: `"r"` de **r**ecuperar (só lê, e o arquivo tem de existir); `"w"` de ***wipe***, "limpar" (zera o arquivo e escreve do começo); `"a"` de **a**crescentar (continua do final). O `"a"` dos arquivos faz o mesmo que o `append` das listas.

Atenção

O modo `"w"` apaga o conteúdo **no momento em que o arquivo é aberto**, mesmo que você não escreva nada depois. Abrir com `"w"` um arquivo que você queria apenas consultar destrói os dados.

3. Escrevendo: write

A função `write` grava o conteúdo no arquivo, com duas exigências:

- o valor precisa ser uma **string**. Números precisam ser convertidos, com `str()` ou dentro de uma f-string;
- ela **não pula linha sozinha**, diferente do `print`. Para separar os dados em linhas, escrevemos o caractere `"\n"` ao final.

```
arquivo = open("dados.txt", "w", encoding="utf-8")

for i in range(3):
    arquivo.write(f"{i}\n")

arquivo.close()
```

Conteúdo de `dados.txt` depois da execução:

```
0
1
2
```

Macete

`write` não é `print`. O `print(a, b)` aceita vários valores de qualquer tipo, coloca espaço entre eles e pula linha no final. O `write` aceita **uma única string** e grava **exatamente** o que recebeu. A f-string resolve as duas exigências de uma vez: `f"{valor}\n"` converte o número e já coloca a quebra de linha.

4. Fechando: close e with

Fechar não é um detalhe:

- no modo de escrita, sem fechar, os dados podem **não chegar ao disco**;
- o arquivo pode ficar bloqueado para outros acessos;
- cada arquivo aberto consome uma vaga do limite do sistema.

E se acontecer um erro antes da chamada a `close`? O arquivo fica aberto. O gerenciador de contexto `with` resolve isso: ele garante que o arquivo será fechado automaticamente ao fim do bloco, mesmo que ocorra um erro no meio do caminho.

```
with open("dados.txt", "w", encoding="utf-8") as arquivo:
    for i in range(3):
        arquivo.write(f"{i}\n")

print("Arquivo salvo!")
```

```
Arquivo salvo!
```

Macete

Leia `with open(...) as arquivo:` como "abra o arquivo, chame-o de `arquivo` e, quando o bloco indentado acabar, feche". Não existe chamada a `close`. Tudo o que usa o arquivo precisa estar **indentado** dentro do `with`; o que vem depois, sem indentação, já encontra o arquivo fechado.

5. Lendo um arquivo

Para ler, abrimos no modo `"r"` e percorremos o arquivo **linha a linha** com `for`.

```
with open("dados.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        print(linha)
```

A saída tem uma linha em branco depois de cada valor: o 0, uma linha vazia, o 1, uma linha vazia, o 2, uma linha vazia. Isso acontece porque cada linha lida **inclui o** `"\n"` do final, e o `print` ainda pula outra linha. A função `strip` remove esses espaços e quebras:

```
with open("dados.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        print(linha.strip())
```

```
0
1
```

```
2
```

Atenção

Tudo o que vem de um arquivo de texto chega como string. A linha `"12"` não é o número 12. Para fazer contas, converta com `int()` ou `float()`. Sem converter, `"1" + "2"` resulta em `"12"`, e não em 3.

Macete

Receita de leitura em três verbos: **leu, limpou, converteu**. Leu a linha (`for linha in arquivo`), limpou (`linha.strip()`), converteu se for número (`int(...)`). Faça isso sempre, nessa ordem.

6. O arquivo precisa de um formato

Um arquivo de texto é só uma sequência de caracteres. O significado vem do **formato combinado**: quem grava e quem lê precisam concordar com ele. Os slides trabalham três formatos.

Formato 1: um dado por linha (listas)

```python
placas = ["ABC1D23", "XYZ9K87", "QWE4R56", "JKL2M34"]

with open("patio.txt", "w", encoding="utf-8") as arquivo:
    for placa in placas:
        arquivo.write(f"{placa}\n")

recuperadas = []

with open("patio.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        recuperadas.append(linha.strip())

print(recuperadas)
```

```
['ABC1D23', 'XYZ9K87', 'QWE4R56', 'JKL2M34']
```

Formato 2: campos separados por vírgula (dicionários)

Cada linha tem dois campos, como `Caneta,15`. Na leitura, `split(",")` devolve uma lista com as partes da linha.

```python
estoque = {"Caderno": 10, "Borracha": 25, "Lapis": 50}

with open("estoque.txt", "w", encoding="utf-8") as arquivo:
    for chave, valor in estoque.items():
        arquivo.write(f"{chave},{valor}\n")

estoque_recuperado = {}

with open("estoque.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        partes = linha.strip().split(",")
```

```
        produto = partes[0]
        quantidade = int(partes[1])
        estoque_recuperado[produto] = quantidade

print(estoque_recuperado)
```

```
{'Caderno': 10, 'Borracha': 25, 'Lapis': 50}
```

Macete

Decore a linha `partes = linha.strip().split(",")`. Ela transforma `"Caneta,15\n"` em `["Caneta", "15"]`: o `strip` tira o `"\n"` e o `split` corta na vírgula. Depois, `partes[0]` é o nome e `int(partes[1])` é o número. O separador pode ser outro (o professor já usou `";"` em lista): troque dos **dois** lados, na escrita e no `split`.

Formato 3: JSON

JSON é um formato padronizado, entendido por praticamente qualquer linguagem. Sua estrutura é praticamente a mesma de um dicionário Python. Nos slides, o arquivo é montado **linha a linha**, à mão:

```
estoque = {"Caderno": 10, "Borracha": 25, "Lapis": 50}

itens = list(estoque.items())
total = len(itens)

with open("estoque.json", "w", encoding="utf-8") as arquivo:
    arquivo.write("{\n")

    for i in range(total):
        chave, valor = itens[i]
        linha = f'    "{chave}": {valor}'

        if i < total - 1:
            linha = linha + ","

        arquivo.write(linha + "\n")

    arquivo.write("}\n")
```

Conteúdo de `estoque.json`:

```
{
    "Caderno": 10,
    "Borracha": 25,
    "Lapis": 50
}
```

Atenção

O detalhe da última linha: no formato JSON, não existe vírgula depois do último item. É por isso que o laço usa o índice e testa `if i < total - 1` antes de acrescentar a vírgula. Repare também que a f-string usa aspas **simples** por fora, porque o texto gravado contém aspas duplas.

Para ler de volta, limpamos cada linha e separamos a chave do valor:

```
recuperado = {}

with open("estoque.json", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        linha = linha.strip(" \n,{}")

        if not linha:
            continue

        partes = linha.split(":")
        chave = partes[0].strip('"')
        valor = int(partes[1])
        recuperado[chave] = valor

print(recuperado)
```

```
{'Caderno': 10, 'Borracha': 25, 'Lapis': 50}
```

Três detalhes importantes desse código:

- `strip(" \n,{}")` remove, das **pontas** da linha, todos os caracteres passados por parâmetro: espaço, quebra de linha, vírgula e chaves;
- uma linha vazia (`""`) é avaliada como **falsa**; por isso o `if not linha` detecta as linhas que só tinham `{` ou `}`;
- `continue` interrompe **apenas a iteração atual** e avança para a próxima linha; o `break` interromperia o laço inteiro.

Macete

Acompanhe uma linha passo a passo. A linha lida é `"Caderno": 10,` seguida de `"\n"`. Depois de `strip(" \n,{}")` fica `"Caderno": 10`. O `split(":")` separa em `"Caderno"` (ainda com as aspas duplas) e `10` (com um espaço na frente). O `strip('"')` tira as aspas da chave, e o `int` aceita o número mesmo com o espaço.

7. Os padrões que resolvem quase tudo

Nos três formatos, a estrutura é a mesma: **abrir no modo adequado, processar linha a linha, fechar com with**. O que muda é o formato combinado dentro do arquivo.

Padrão 1: ler do teclado e gravar (até "fim")

```
with open("nomes.txt", "w", encoding="utf-8") as arquivo:
    nome = input()

    while nome != "fim":
        arquivo.write(nome + "\n")
        nome = input()
```

Para **acrescentar** aos nomes de execuções anteriores, em vez de substituir, basta trocar `"w"` por `"a"`.

Padrão 2: contar e acumular

```
with open("notas.txt", "w", encoding="utf-8") as arquivo:
    arquivo.write("Ana,8.5\nBruno,4.0\nCarla,6.0\n")

soma = 0
quantidade = 0

with open("notas.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        partes = linha.strip().split(",")
        soma += float(partes[1])
        quantidade += 1

print(quantidade, soma / quantidade)
```

```
3 6.166666666666667
```

Padrão 3: filtrar de um arquivo para outro

Dois arquivos abertos, um em cada modo: lê do primeiro e grava no segundo apenas o que interessa.

```
with open("notas.txt", "r", encoding="utf-8") as entrada:
    with open("aprovados.txt", "w", encoding="utf-8") as saida:
        for linha in entrada:
            partes = linha.strip().split(",")

            if float(partes[1]) >= 6:
                saida.write(linha)

with open("aprovados.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        print(linha.strip())
```

```
Ana,8.5
Carla,6.0
```

Como a `linha` original já termina com `"\n"`, ela pode ser gravada diretamente, mantendo o mesmo formato.

Padrão 4: funções salvar e carregar

```
def salvar_lista(valores, nome_arquivo):
    with open(nome_arquivo, "w", encoding="utf-8") as arquivo:
        for valor in valores:
            arquivo.write(f"{valor}\n")


def carregar_lista(nome_arquivo):
    valores = []

    with open(nome_arquivo, "r", encoding="utf-8") as arquivo:
        for linha in arquivo:
            valores.append(linha.strip())
```

```
    return valores


salvar_lista([5, 4, 3], "valores.txt")
print(carregar_lista("valores.txt"))
```

```
['5', '4', '3']
```

Repare: gravamos números e recuperamos **strings**. Para que a lista carregada seja igual à original, a função precisaria fazer `valores.append(int(linha.strip()))`.

8. Erros mais comuns

Atenção

- **Esquecer o `"\n"`:** tudo é gravado em sequência, na mesma linha.
- **Passar número para o write:** `arquivo.write(10)` dá erro. Use `str(10)` ou uma f-string.
- **Usar "w" querendo acrescentar:** o conteúdo anterior é apagado. O modo certo é `"a"`.
- **Abrir com "r" um arquivo que não existe:** erro. Os modos `"w"` e `"a"` criam o arquivo; o `"r"` não.
- **Esquecer o strip:** as strings ficam com `"\n"` no final, e comparações como `linha == "fim"` falham.
- **Esquecer de converter:** somar ou comparar textos em vez de números.
- **Vírgula depois do último item do JSON.**
- **Usar o arquivo fora do with:** depois do bloco, o arquivo já está fechado.

9. Questões de exemplo resolvidas

Questão 1 — O que o arquivo contém no final? (quiz dos slides)

O arquivo `turma.txt` já contém os nomes Ana e Bruno, um por linha. O que ele contém depois da execução do código abaixo?

```
with open("turma.txt", "w", encoding="utf-8") as arquivo:
    arquivo.write("Ana\nBruno\n")

with open("turma.txt", "w", encoding="utf-8") as arquivo:
    arquivo.write("Carla\n")

with open("turma.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        print(linha.strip())
```

```
Carla
```

Resposta: apenas Carla. O primeiro `with` só recria a situação inicial. O segundo abre no modo `"w"`, que apaga o conteúdo anterior. Para manter os nomes antigos, o modo correto seria `"a"`, e o arquivo terminaria com Ana, Bruno e Carla.

Questão 2 — Como fica o arquivo? E a lista? (quiz dos slides)

```python
with open("numeros.txt", "w", encoding="utf-8") as arquivo:
    for i in range(3):
        arquivo.write(f"{i}")

with open("numeros.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        print(linha)
```

```
012
```

Resposta: uma única linha, 012. Sem o `"\n"`, tudo é gravado em sequência. Agora a versão correta da escrita, seguida da leitura para uma lista:

```python
with open("numeros.txt", "w", encoding="utf-8") as arquivo:
    for i in range(3):
        arquivo.write(f"{i}\n")

valores = []

with open("numeros.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        valores.append(linha.strip())

print(valores)
```

```
['0', '1', '2']
```

Resposta: `['0', '1', '2']`, com aspas. Não é `[0, 1, 2]`, porque tudo o que vem do arquivo é string, nem `['0\n', '1\n', '2\n']`, porque o `strip` removeu as quebras.

Questão 3 — Média das notas e arquivo de aprovados (fixação 7 e 8)

O arquivo `notas.txt` tem, em cada linha, o nome de um aluno e a sua nota separados por vírgula. Exiba a média das notas e grave em `aprovados.txt` apenas os alunos com nota maior ou igual a 6, no mesmo formato.

Raciocínio. Entrada: o arquivo. Processamento: para cada linha, limpar, separar na vírgula e converter a nota; acumular para a média; guardar os aprovados. Saída: a média na tela e o novo arquivo. Aqui os aprovados são guardados em uma lista e gravados depois, o que mantém um arquivo aberto por vez.

```python
with open("notas.txt", "w", encoding="utf-8") as arquivo:
    arquivo.write("Ana,8.5\nBruno,4.0\nCarla,6.0\nDavi,5.5\n")

soma = 0
quantidade = 0
aprovados = []

with open("notas.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
```

```
        partes = linha.strip().split(",")
        nome = partes[0]
        nota = float(partes[1])

        soma += nota
        quantidade += 1

        if nota >= 6:
            aprovados.append(f"{nome},{nota}\n")

with open("aprovados.txt", "w", encoding="utf-8") as arquivo:
    for registro in aprovados:
        arquivo.write(registro)

print("Media:", soma / quantidade)
print("Aprovados:", len(aprovados))
```

```
Media: 6.0
Aprovados: 2
```

O primeiro `with` apenas cria o arquivo de teste. Na prova, ele já existiria.

Questão 4 — Leitura de código

O arquivo `registros.txt` contém as linhas `A;8`, `B;5` e `A;4`. Escreva a saída do programa.

```
with open("registros.txt", "w", encoding="utf-8") as arquivo:
    arquivo.write("A;8\nB;5\nA;4\n")

totais = {}

with open("registros.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        partes = linha.strip().split(";")
        setor = partes[0]
        valor = int(partes[1])

        if setor not in totais:
            totais[setor] = 0

        totais[setor] = totais[setor] + valor
        print(setor, totais[setor])

print(totais)
```

Raciocínio. É a mistura que o professor gosta de cobrar: leitura de arquivo com contagem em dicionário. O separador aqui é `";"`. Acompanhe linha a linha: A recebe 8; B recebe 5; A já existe e passa a 12.

```
A 8
B 5
A 12
{'A': 12, 'B': 5}
```

10. Cola rápida

Quero...	**Escrevo...**
abrir para ler	`with open("a.txt", "r", encoding="utf-8") as arq:`
abrir para escrever do zero	`with open("a.txt", "w", encoding="utf-8") as arq:`
abrir para acrescentar	`with open("a.txt", "a", encoding="utf-8") as arq:`
gravar uma linha	`arq.write(f"{valor}\n")`
gravar dois campos	`arq.write(f"{chave},{valor}\n")`
percorrer as linhas	`for linha in arq:`
limpar a linha	`linha.strip()`
separar os campos	`linha.strip().split(",")`
texto para número	`int(texto)` ou `float(texto)`
pular linha vazia	`if not linha: continue`